

Kubernetes Notes

Your Name

Date:

Contents

1	Notes info and Introduction	5
2	Core Concepts	7
2.1	Kubernetes Architecture	7
2.1.1	What is installed on a system when installing Kubernetes?	7
2.2	Docker-vs-ContainerD	9
2.3	Pods	9
2.3.1	Muti-Container Pods	10
2.4	YAML Basics	10
2.5	Pods with YAML	10
2.5.1	YAML in Kubernetes	10
2.5.2	ReplicatSets	11
3	Configuration	19
3.1	Define build and modify container images	19
3.2	Commands and Arguments in Docker	20
3.3	Commands and Arguments in Kubenetes	20
3.4	Envrinoment Variables	21
3.5	ConfigMaps	21
3.6	Secrets	23
3.7	Pre-requisite for Kubernetes - Security in Docker	25
3.8	Security Contexts	27
3.9	Resource Requirements	27
4	Multi-container pods	31
4.1	Multi-container pods	31
4.2	Multi-container pods design patterns	31
4.3	Init containers	31
5	Observability	33
5.1	Readiness and Liveness Probes	33
5.2	Liveness Probes	33
5.3	Logging	33
5.4	Monitor and Debug application	33

6	Pod Design	35
6.1	Labels, Selectors and Annotations	35
6.2	Rolling updates and Rollbacks in Deployments	35
6.3	Updating a deployment	35
6.4	Deployment strategy - Blue Green	35
6.5	Deployment strategy - Canary	35
6.6	Jobs	35
6.7	CronJobs	35
7	Services and Networking	37
7.1	Network Policies	37
7.2	Developing network policies	37
7.3	Ingress networking	37
8	State Persistence	39
8.1	Introduction to Docker Storage	39
8.2	Storage in Docker	39
8.3	Volume Driver Plugins in Docker	39
8.4	Volumes in Kubernetes	39
8.5	Persistent Volumes	39
8.6	Persistent volume claims	39
8.7	Using PVCs in Pods	39
8.8	Storage Classes	39
8.9	Why Stateful Sets	39
8.10	Stateful sets introduction	39
8.11	Headless Services	39
8.12	Storage in StatefulSets	39
9	Security	41
9.1	Kubernetes Security Primitives	41
9.2	Authentication	41
9.3	KubeConfig	41
9.4	API Groups	41
9.5	Authorization	41
9.6	Role based access controls	41
9.7	Cluster Roles	41
9.8	Admission Controllers	41
9.9	Validating and Mutating Admission Controllers	41
9.10	API Versions	41
9.11	API Deprecations	41
9.12	Custom Resource Definition	41
9.13	Custom Controllers	41
9.14	Operator Framework	41
10	Helm Fundamentals	43
10.1	Helm Introduction	43
10.2	Install Helm	43
10.3	Helm concepts	43

11 Kustomize	45
11.1 Kustomize Problem Statement and Ideology	45
11.2 Kustomize vs Helm	45
11.3 Kustomization yaml file	45
11.4 Kustomization Output	45
11.5 Kustomization ApiVersion and Kind	45
11.6 Managing Directories	45
11.7 Common Transformers	45
11.8 Image Transformers	45
11.9 Patches Intro	45
11.10 Different Types of Patches	45
11.11 Patches Dictionary	45
11.12 Patches list	45
11.13 Overlays	45
11.14 Components	45

Chapter 1

Notes info and Introduction

Purpose of notes There purpose of these notes are to gain an understanding of Docker and have reference material in case I decide to sit the DCA exam.

Chapter 2

Core Concepts

2.1 Kubernetes Architecture

Nodes A node is a machine, physical or virtual upon which Kubernetes is installed. This worker machine is where containers will be launched by Kubernetes.was known as minions in the past... These terms may be used interchangeably.

Consider a node failing.....application will go down.....hence we need multiple nodes.

Cluster A cluster is a group of nodes grouped together. This way when one node fails, your application is still accessible from the other nodes. Having multiple nodes helps in sharing the load as well.

Master node Things to consider - What is responsible for managing the cluster - Where is the information about members of the cluster stored? - How are nodes monitored? - When a node fails, how do you move the workload of the failed node to another worker node?

This is the purpose of the master. The master is another node with Kubernetes installed in it and is configured as a master. The master watches over the nodes in the cluster and is responsible for the actual orchestration of containers on the worker nodes.

2.1.1 What is installed on a system when installing Kubernetes?

- API Server
- etcd
- Kubelet
- Container Runtime
- Controller
- Scheduler

API Server

An API server which acts as the frontend for Kubernetes. The user management devices, command line interfaces all talk to the API server to interact with the Kubernetes cluster.

etcd keystore

Etcd is a distributed, reliable key value store used by Kubernetes to store all data used to manage the cluster. The way to think of it is, when you have a multiple nodes and masters in your cluster, etcd stores all that information on all the nodes in the cluster in a distributed manner Etcd is responsible for implementing locks within the cluster to ensure that there are no conflicts between the masters,

The Scheduler

is responsible for distributing work or containers across multiple nodes. It looks for newly created containers and assigns them to nodes.

Controllers

The "brain" behind orchestration. They are responsible for noticing and responding when nodes, containers or endpoints go down. The controllers decide whether to bring up new containers.

Container Runtime

They are responsible for noticing and responding when nodes, containers or endpoints runtime is the underlying software that is used to run containers. An example is Docker, but alternatives exist.

Kubelet

This is the agent that runs on each node in the cluster. The agent is responsible for making sure that the containers are running on the nodes as expected.

There are two types of server, masters and a worker along with a setup of components which make up Kubernetes.

How does one server become a master and the other the slave.

The worker node, or minion as it is also known, is where the containers are hosted.

The master has the Kube API server and that is what makes it a master. Similarly the worker nodes have the Kubelet agent that is responsible for interacting with master to provide both health information of the worker node and carry out actions requested by the master on the worker nodes.

All the information gathered is stored in a key value store on the master. The key value store is based on the popular etcd framework.

The master also has the control manager and the scheduler.

Kube command line or kubectl or kube control

The kube control tool, is used to deploy and manage applications on Kubernetes cluster to get cluster information, to get the status of other nodes in the cluster. And to manage other things. The kubectl cluster info command is used to view information about the cluster and the kubectl get nodes command is used to list all the nodes as a part of the cluster.

2.2 Docker-vs-ContainerD

Initially kubernetes was the container runtime for docker only. Support for other container runtimes was added. This was done via an interface called Container Runtime Interface or CRI.

CRI allows any vendor to work as a container runtime for Kubernetes, as long as they adhere to the OCI standards. OCI stands for the open container initiative and it consists of an image spec and runtime spec.

Image spec means the specifications on how an image should be built. An image spec defined the specifications on how an image should be built and the runtime spec defined the standards on how nay container runtime should be developed.

Keeping the OCI standards in mind. Anyone can build a container runtime that can be used by anybody to work with Kubernetes Note: Docker wasn't built for CRI. To maintain support between kubernetes and Docker, Kubernetes introduced dockershim. Docker runs without CRI.

Docker isn't just a container runtime along. Docker consists of multiple tools that are put together. - Docker CLI, the Docker API and the build tools that help in building images. There was support for volumes or security. There is a container runtime called run C and that managed a container called container D Container D can be used as a runtime on its own separated from Docker.

...dockershim and docker support was removed from kubernetes but docker follows the OCI standards now.

Container D is a part of Docker but is a separate project on its own. You can run container D without having to install Docker itself. If you don't need dockers other features you could ideally just install container D alone.

CLI - ctr

CLI - nerdctl

nerdctl provides a docker like CLI for containerd nerdctl supports Docker compose nerdctl supported newest features in containerd - Encrypted container images - Lazy pulling - P2P image distribution - image signing and verifying - Namespaces in Kubernetes

CLI-crictl

crictl provides a CLI for CRI-compatible container runtimes installed separately used to inspect and debug container runtimes not to create container ideally works across different runtimes

2.3 Pods

Containers are encapsulated into a Kubernetes object know as pods. A pod is a single instance of an application. A pod is the smallest object you can create in Kubernetes.

Can increase the number of pods to scale the application. Can add a new node and add pods to that if the current nodes do not have capacity.

Pods usually have a 1 to 1 relationship with You do not add containers to a single pod to scale your applicaiton.

2.3.1 Muti-Container Pods

You can have multiple containers in the same pod but they will not be the same kind. Example: helper container. The two containers in the same pod can communicate with each other over localhost. They can easily share the same storage space and the same network space.

2.4 YAML Basics

2.5 Pods with YAML

Kubernetes uses YAML files as inputs for the creation of objects such as pods, replicas and deployments, services.

2.5.1 YAML in Kubernetes

- `apiVersion`: This is the version of the Kubernetes API we are using to create the object. Must use the correct API version for the objects we are trying to create. See other options.
- `kind`: This refers to the type object we are trying to create. Options are Pod, Service, ReplicaSet and Deployment.
- `metadata`: This is data about the object like its name, labels, etc.
- `spec`:

Listing 2.1: Pod commands

```
kubectl create -f pod-definition.yml
kubectl get pods
kubectl describe pod myapp-pod
```

Listing 2.2: Example YAML Configuration

```
database:
  host: localhost
  port: 5432
  username: admin
  password: secret

features:
  logging: true
  cache: false

servers:
  - app01.example.com
  - app02.example.com
```

2.5.2 ReplicatSets

Replication Controller

The replication controller helps us run multiple instances of a single pod in the Kubernetes cluster allowing for High Availability. This can be done with a single pod. The replication controller can help by automatically bring up a new pod when the existing one fails. The replication controller ensures that the specified number of pods are running at all times.

Another reason we need replication controllers is to create multiple pods to share the load across them.

The replication controller allows us to create multiple pods to share the load across them. The replication controller spans across multiple nodes in the cluster. It allows for the application to be scaled when the demand increases.

Note: there are two similar terms. Replications Controller and Replica Set. Both have the same purpose, but they are not the same. Replication controller is the older technology that is being replaced by the replicat set.

Replicat set is the new recommended way to set up replication. There are minor differences in the way each works.

How to create a replication controller

Start by creating a rc-definition.yml YAML file. This will have the standard info as any Kubernetes file. API, version, kind, metadata and spec. The API version is specific to what we are creating. The kind is set to replication controller. Add a name under metadata. Add labels and assign values. Add the spec to the YAML. The spec defines what is inside the object we are creating. Create a template section under the pod template to be used by the replication controller to create replicas.

How is the pod template defined? Use pod definition file metadata.

Listing 2.3: Replication controller

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      container:
        - name: nginx-container
          image: nginx
  replicas: 3
```

Listing 2.4: Pod commands

```
kubectl create -f rc-definition.yml
```

```
kubectl get replicationcontroller
kubectl get pods
```

Listing 2.5: Replication Set

```
apiVersion: apps/v1
kind: ReplicationSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metedata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      container:
        - name: nginx-container
          image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

The selector section helps the replica set identify what pods fall under it. Replica sets can also manage pods that were no created as a part of the replica set creation. Example, pods created before the creation of the replica sets that match the labels specified in the selector. The replicaset will also take those pods into consideration when creating the replicas.

The selector is one of the major differences between replication controller and replica set. The selector is not a required field in case of a replication controller, but it is still available.

Match label selector simply matches the labels specified under it to the labels on the pods.

Labels and selectors

Uses of replica sets. Use them to monitor existing pods if they have already been created.

The role of the the replica set is to monitor the piods and if any of them were to fail, deploy new ones. The replica set is a process that monitors the pods.

Labelling the pods is handy since we can now use these as a filter for replica set under the selector section, we use the match labels filter and provide the same filter we used while creating the pods.

Scaling with a replica set

One way is to update the number of replicas in the definition file to six replicas the run kubectl's apply command to sspecifiy the same file using the f parameter and the will update the replica set to have six replicas.

Another way is to run the kube control scal command. Use the replicas parameter to provide the number of replicas and specify the same file as input. - You can either input the definition file

or provide the replica set name in the type name format. However, remember that using the file name as input will not result in the number of replicas being updated automatically in the file. — In other words the number of replicas in the replica set definition file will still be three, even though via this command it was scaled to six.

There are also options available for automatically scaling the replica set based on load, but that is an advanced topic.....

Listing 2.6: Pod commands

```
kubectl create -f replicaset-definition.yml
kubectl get replicaset
kubectl delete replicaset myapp-replicaset
kubectl replace -f replicaset-definition.yml
kubectl scale --replicas=6 -f replicaset-definition.yml
```

Deployment

Each container is encapsulated inside a pod. Multiple pods are deployed using replication controllers or replica sets.

Higher in the hierarchy of Kubernetes, above replica sets are deployments

Listing 2.7: Replication Set

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      container:
        - name: nginx-container
          image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

Listing 2.8: Pod commands

```
kubectl create -f deployment-definition.yml
kubectl get deployments
kubectl get replicaset
kubectl get pods
kubectl get all
```

Namespaces

Kubectl Explain Command

Pods, deployments and clusters are deployed within a namespace.

Kubernetes creates a default namespace when the cluster is setup and these are created under a namespace called kubesystem. The kube-public namespace is where resource that should be made available to all users are created.

When a kubernetes cluster grows to production/enterprise level you will want to consider using namespaces.

You can create your own namespaces as well. Example, if you want to use the same cluster for both dev and production environments but at the same time isolate the resources between them. You can create a different namespace for each of them.

That way while working in the dev environment, you don't accidentally modify resources in production.

Each of these namespaces can have its own set of policies that define who can do what.

You can also assign a quota of resources to each of these namespaces. That way, each namespace is guaranteed a certain amount and does not use more than its allowed limit.

Resources within a default namespace can refer to each other by their names.

You can make sure that the pod gets created into the development environment all the time by moving the namespace into the pod definition file.

Listing 2.9: Replication Set

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: myapp-pod
  namespace: dev
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Listing 2.10: Pod commands

```
kubectl get pods
kubectl get pods --namespace=kube-system

kubectl create -f pod-definition.yml
kubectl create -f pod-definition.yml --namespace=dev
```

Create Namespace

Listing 2.11: Replication Set

```
apiVersion: apps/v1
kind: Namespace
```

```
metadata:
  name: dev
```

Listing 2.12: Pod commands

```
kubectl create -f namespace-dev.yml
kubectl create namespace dev
kubectl get pods --namespace=dev
#Used to change context for namespace command
kubectl config set-context $(kubectl config current-context) --namespace=dev

kubectl get pods
kubectl get pods --namespace=default
kubectl get pods --namespace=prod

# view pods in all namespaces
kubectl get pods --all-namespaces
```

Resource Quota

Listing 2.13: Replication Set

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev

spec:
  hard:
    pods: "10"
    request.cpu: "4"
    requests.memory: 5Gi
    limits.cpu: "10"
    limits.memory: 10Gi
```

Listing 2.14: Pod commands

```
kubectl create -f compute-quota.yaml
```

Services

Kubernetes services enable communication between various components within and outside of the application. These services help connect application together with other applicaitons or users.

Example to consider. Application has a group of pods running variaous sections such as group for running back end processes, a group for frontend services and a gruop connecting to an external data source.

Services enabled the front end applicaiton to made avaiable to end users to end users. It helps communication between backend and fronend pods and helps in establishing connectivity to an external data source.

Services snable loose coupling between microservices in out application.

A kubernetes service is an object just like pods, replica sets or deployments that we worked with before. One of its use cases is to listen to a port on the node and forward requests on that port to a port on the pod running the web application.

This type of service is known as a node port services, because the service listens to a port on the node and forwards the request to the pods.

Service Types

There are different types of services.

The first type is the node port where the service makes an internal port accessible on a port on the node.

The second is cluster IP and in this case the service creates a virtual IP inside the cluster to enable communication between different services, such as a set of front end servers and a set of backend servers.

The third type is a load balancer, where it provisions a load balancer for our applicaiton in supported cloud providers.

Node Port

There are three ports involved. The port on the pod where the actual web server is running. This is referred to as the target port because that is where the service forwards the request. The second port is the port on the service. Referred to as the port. This service is in fact like a virtual server inside the node, inside the cluster. It has its own IP address and that IP address is called the cluster IP of the service. Finally, we have the port on the node itself which we can use to access the web server externally. -¿ this is known as the node port. Node ports can only be in a valid range which is by default from 30,000 to 32,767.

How to create a service

Use a definition file to create a service. The high level structure of the file remains the same as before.

Listing 2.15: Replication Set

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service

spec:
  type: NodePort
  ports:
    - targetPort: 80 #Note that this is an array - you can have multiple p
      port: 80 #Out of all of these this is the mandatory port
      nodePort: 30008
  selector: # Use to identify the pod
    app: myapp
    type: front-end
```

If you don't provide a target port, it is assumed to be the same as port. Also, if you don't provide a node port, a free port in the valid range between 30,000 and 32,767 is automatically allocated.

Listing 2.16: Pod commands

```
kubectl create -f service-definition.yml
kubectl get services
# We can now this port to access the web service using Curl or a web browser
curl http://192.168.1.2:30008
```

Requests are randomly routed to pods.... The service acts as a built-in load balancer to distribute load across different pods

In this case we have web applications in pods on separate nodes in a cluster. Kubernetes creates a service which spans all the nodes in the cluster.

They can be accessed using curl with the correct IP address and port.

To summarise in either case the service is created exactly the same without having to do any additional steps during the service creation.

When pods are removed or added, the service is automatically updated, making it highly flexible and adaptive. Once created, you won't typically have to make any additional configuration changes.

Services - Cluster IP

A full stack web application typically has different kinds of pods hosting different parts of an application. You may have a number of pods running a frontend web server, another set of pods running a backend server, a set of pods running a key value store like redis etc.

They will need to communicate with each other.

What is the right way for them to communicate?

The pods all have an IP address assigned to them but these IPs as we know are not static. These pods can go down any time and new pods are created all the time.

You cannot rely on these IP addresses for internal communication between the application.

A Kubernetes service can help us group the pods together and provide a single interface to access the pods in the group.

Example: A service created for the backend pods will help group all the backend pods together and provide a single interface for other pods to access this service. The requests are forwarded to one of the pods under the service randomly.

Similarly create additional service for readings and allow the backend pods to access the reading systems through the service.

This enables us to easily and effectively deploy a microservices based application on a Kubernetes cluster.

Each layer can now scale or move as required without impacting communication between various services.

Each service gets an IP and name assigned to it inside the cluster and that is the name that should be used by other paths to access the service.

This type of service is known as the cluster IP

Listing 2.17: Cluster IP service definition

```
apiVersion: v1
kind: Service
metadata:
  name: back-end
spec:
  type: ClusterIP # This is default
```

```
ports:
  - targetPort: 80 # Where the backend is exposed
    port: 80 # Where the service is exposed
selector: # To Link a service to a set of ports
  app: myapp
  type: back-end
```

Listing 2.18: Commands

```
kubectl create -f service-definition.yml
kubectl get services
```

Kubectl Command

SEE DOCS kubectl cheat sheet and kubectl overview

Listing 2.19: Commands

```
kubectl api-resources
kubectl explain pods #top level
kubectl explain pods --recursive # Easier to create YAML files out of this
```

Chapter 3

Configuration

3.1 Define build and modify container images

Why create your own image? BEcause you cannot find a component or a service that you want to use as a part of your application on Docker Hub already or your team decided that you're development will be dockerized for ease of shipping and deployment.

How to create my own image?

First we need understand what we are containerizing or what application we are creating and image ofr and how the applicaiton is built. Start by thinking if you want to deploy the application manually.

- OS - Ubuntu
- Update apt repo
- Install dependencies using apt
- Install Python dependencies using pip
- Copy source code to /opt folder
- Run the web server using "flask" command

Listing 3.1: Example docker file

```
FROM Ubuntu # Base OS or another image

RUN apt-get update # Install all dependencies
RUN apt-get install python

RUN pip install flask
RUN pip install flask-mysql

COPY . /opt/source-code # Copy source code to a location

ENTRYPOINT FLASK_APP=/opt/source-code/app.py flask run # Specify Entrypoint -
```

Listing 3.2: Commands

```
docker build Dockerfile -t username/my-custom-app # The above command will create an image local
docker push username/my-custom-app #This makes the image available on the local docker hub
```

Dockerfile

A docker file is a text file written in a specific format. `{Instruction} {Argument}`

Each of these instruct docker to perform a specific file while creating the image.

When docker builds the images it builds this in a layered architecture. Each line of instruction creates a new layer in the docker image with just the changes from the previous layer.

When running the "docker build ." command you can see the various steps involved and the result of each task. A docker build can be restarted from a step incase a step fails or if you were to add new steps in the build process you wouldn't have to start again. Previous layers are reused from a cache.

This helpful especially when you update source of your application, as it may change more frequently. Only layers above the updated layers needs to be rebuilt.

Everything can be containerized including browsers like firefox etc and utilities like curl.

3.2 Commands and Arguments in Docker

note: not required for cert but useful in usage

Unlike virtual machine, containers are no meant to host an operating system Containers are meant to run a specific task or process, such as to host an instance of a web server, or application server or a database. Or simply to carry out some kind of computation or analysis.

Once the task is complete the container exits. A container only lives as long as the process inside it is alive.

If the web service inside the container is stopped or crashses the container exists.

Command line arguments can be appended to the entry point.

3.3 Commands and Arguments in Kubenetes

Listing 3.3: Cluster IP service definition

```
apiVersion: v1
kind: Pod
metadata:
  name: ubuntu-sleeper-pod
spec:
  containers:
    - name: ubuntu-sleeper
      image: ubuntu-sleeper
      command: ["sleep.2.0"] # Corresponds to entry point in docker file
      args: ["10"] # Corresponds to CMD in docker file
```

Listing 3.4: Commands

```
docker run -r APP_COLOR=pink simple-webapp-color
```

3.4 Environment Variables

Listing 3.5: Example pod definition file with yml

```
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
spec:
  container:
  - name: simple-webapp_color
    image: simple-webapp-color
    ports:
    - containerPort: 8080
    env: # These are plain key values
    - name: APP_COLOR
      value: pink
```

There are other ways of setting these environment values. Plain key values and secrets

Listing 3.6: Example pod definition file with yml

```
env:
  - name: APP_COLOR # Config map
    valueFrom:
      configMapKeyRef:

/////////
env: # Secrets
  - name: APP_COLOR
    valueFrom:
      secretKeyRef:
```

3.5 ConfigMaps

used to manage environment variable centrally

Listing 3.7: Example pod definition file with yml

```
apiVersion: v1
kind: pod
metadata:
  name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
    - containerPort: 8080
    envFrom:
    - configMapRef:
        name: app-config
```

Listing 3.8: Example pod definition file with yaml

```
APP_COLOR: blue
APP_MODE: prod
```

Create the ConfigMap Inject the config map into the pod

There are two ways of creating a config map.

The imperative way without using a config map definition file - `kubectl create configmap` The declarative way by using a config map definition file.

If you don't wish to create a config map definition file, you just use the `Kubectl` control command

Listing 3.9: Example pod definition file with yaml

```
kubectl create configmap <config-name> \
  --from-literal=APP_COLOR=blue \
  --from-literal=APP_MODE=prod
```

— using a path to file

Listing 3.10: Example pod definition file with yaml

```
kubectl create configmap <config-name> \
  --from-file=<path-to-file>
```

Using declarative approach

Listing 3.11: Example pod definition file with yaml

```
kubectl create -f
```

Listing 3.12: Example pod definition file with yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_COLOR: blue
  APP_MODE: prod
```

Listing 3.13: Example pod definition file with yaml

```
kubectl create -f config-map.yaml
```

Listing 3.14: Viewing config maps

```
kubectl get configmaps
kubectl describe configmaps
```

Example

Listing 3.15: Example pod definition file with yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```
    name: app-config
data:
  APP_COLOR: blue
  APP_MODE: prod
```

Listing 3.16: Example pod definition file with yml

```
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080
    envFrom:
      - configMapRef:
          name: app-config
```

Listing 3.17: Viewing config maps

```
kubectl create -f pod-definition.yaml
```

3.6 Secrets

Listing 3.18: Example pod definition file with yml

```
import os
from flask import Flask

app = Flask(__name__)

@app.route("/")
def main():
    mysql.connector.connect(host='mysql', database='mysql', user='root', password='password')
    return render_template('hello.html', color=fetchcolor())

if __name__ == "__main__":
    app.run(host="0.0.0.0", port="8080")
```

Listing 3.19: Example pod definition file with yml

```
apiVersion:
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_Host: mysql
  DB_User: root
  DB_Password: paswr
```

Note the hardcoded values. This is bad. Can store this in a Config. Use secrets to store this sensitive information. Similar to config maps except that they are stored in an encoded or hashed format

Two steps when working with secrets - Create the secrets - Inject into the pod

Listing 3.20: Example pod definition file with yml

```
DB_Host: mysql
DB_User: root
DB_Password: paswr
```

Listing 3.21: Imperative creation

```
kubectl create secret generic
```

Listing 3.22: Declarative creation

```
kubectl create -f
```

Listing 3.23: Declarative creation

```
kubectl create -f
```

Listing 3.24: Example pod definition file with yml

```
apiVersion:
kind: Secret
metadata:
  name: app-secret
data:
  DB_Host: mysql #This is plain text. Data must be specified in an encoded f
  DB_User: root
  DB_Password: paswr
```

Listing 3.25: Declarative creation

```
kubectl create -f secret-data.yaml
```

Encoding Secrets

Listing 3.26: Example pod definition file with yml

```
DB_Host: mysql #This is plain text. Data must be specified in an encoded f
DB_User: root
DB_Password: paswr
```

Listing 3.27: Declarative creation

```
echo -n 'mysql' | base64
echo -n 'root' | base64
echo -n 'paswr' | base64

kubectl get secrets # view secrets
kubectl describe secrets
kubectl get secret app-secret -o yaml

echo -n 'mysql' | base64 --decode
echo -n 'root' | base64 --decode
echo -n 'paswr' | base64 --decode
```

Listing 3.28: pod-definition.yml

```
apiVersion: v1
kind: pod
metadata:
  name: simple-webapp-color
labels:
  name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
    - containerPort: 8080
    envFrom:
    - secretRef:
        name: app-secret
```

Listing 3.29: secret-data.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
data:
  DB_host: ewafe-
  DB_User: ;afweohiu
  DB_Password: ;oiuhj
```

Listing 3.30: Declarative creation

```
kubectl create -f pod-definition.yaml
```

There are other ways to inject secrets into pods. You can inject single environment variables or inject the whole secret as files in a volume.

If you were to mount the secret as a volume in the pod, each attribute in the secret is created as a file with the value of the secret as its content.

Listing 3.31: secret-data.yaml

```
volumes:
- name: app-secret-volumes
secret:
  secretName: app-secret
```

Listing 3.32: Commands to run from within the container

```
ls /opt/app-secret-volumes
cat /opt/app-secret-volumes/DB_Password
```

If you were to mount the secret as a volume in the pod, each attributed in the secret is created a separate file with the value of the secret as its content.

3.7 Pre-requisite for Kubernetes - Security in Docker

Consider a host with Docker installed on it. This host has a set of its own processes running such as a number of operating system processes, the docker daemon itself, the SSH server, etc.

Unlike virtual machines containers are not completely isolated from their host containers and the hosts share the same kernel.

Containers are isolated using namespaces.

In Linux, the host has a namespace and the containers have their own namespace. All the processes run by the containers are in fact run on the host itself, but in their own namespace.

As far as the Docker container is concerned, it is in its own namespace and it can see its own processes only. — It cannot see anything outside of it or in any other namespace.

For the Docker host, all processes of its own, as well as those in the child namespaces are visible as just another process in the system.

When you list the processes on the host, you see a list of processes. — A process can have different process IDs in different namespaces. — That's how Docker isolates containers within a system.

That is process isolation.

Users in the context of security

The Docker host has a set of users, a root user as well as a number of non-root users. By default, Docker runs processes within containers as the root user. The process is run as the root user. This can be seen via commands both inside the container and outside the container.

If you do not want the process within the container to run as the root user, you may set the users using the `user` option within the `docker run` command and specify the new user ID — The process now runs with the new user ID.

Another way to enforce user security is to have this defined in the Docker image itself at the time of creation.

Root user

Docker implements a set of security features that limits the abilities of the root user within the container. So the root user within the container isn't really like the root user on the host. Docker uses Linux capabilities to implement this. Note: Root users can literally do anything, as can a process run by the root user. It has unrestricted access to the system from modifying files and permissions on files, access controller, creating or killing processes, setting group ID or user ID — Performing network related operations such as binding to network ports, broadcasting on a network, controlling network port system related operations like rebooting the host, manipulating the system clock and others.

All of these are the different capabilities on a Linux system and you can see a full list of this at this location. `/usr/include/linux/capability.h`

You can control and limit what capabilities are made available to a user. By default, Docker runs a container with a limited set of capabilities and so the processes running within the container do not have the privileges to reboot the host or perform other operations that can disrupt the host or other containers running on the same host.

If you wish to override this behaviour and provide additional privileges that what is available you can use

Listing 3.33: Commands

```
docker run --cap-add MAC_ADMIN ubuntu
docker run --cap-drop KILL # drop privileges
docker run --privileged #Run a container with all privileges enabled
```

3.8 Security Contexts

In Kubernetes containers are encapsulated in pods. You may choose to configure the security settings at a container level or at a pod level. If you configure it at both the pod and the container level the settings on the container will override the settings on the pod.

Listing 3.34: secret-data.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  securityContext:
    runAsUser: 1000 # Pod level user id
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["woop", "3768"]
      securityContext:
        runAsUser: 1000 # Container level user id security
        capabilities:
          add: ["MAX_ADMIN"] #Specify a list of capabilities to add to t
```

3.9 Resource Requirements

Each node has a set of CPU and memory resource available. Every pod requires a set of resources to run. Whenever a pod is placed on a node, it consumes the resources available on that node. It is the Kubernetes Scheduler that decides which node a pod goes to. The scheduler takes into consideration the amount of resources required by a pod and those available on the nodes. It identifies the best node to place a pod on.

If a node has no sufficient resources available. The scheduler avoids placing the pod on those nodes. Instead, it places the pod on the one where sufficient resources are available. If there are no sufficient resources available on any of the nodes then the scheduler holds back, scheduling the pod and you will see the pod in a pending state.

If you look at the events using the `kubectl describe pod` command you will see that there is an insufficient CPU command.

You can specify the amount of CPU and memory required for a pod when creating one. This is known as the resource request for a container.

When the scheduler tries to place the pod on a node, it uses these numbers to identify a node which has sufficient amount of resources available.

Example Yaml with resource request

Listing 3.35: pod-definition.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
```

```

    labels:
      name: simple-webapp-color
  spec:
    containers:
    - name: simple-webapp-color
      image: simple-webapp-color
      ports:
        - containerPort: 8080
      resources:
        requests:
          memory: "4gi"
          cpu: 2

```

When the scheduler gets a request to place this pod, it looks for a node that has this amount of resources available. And when the pod gets placed on a node, the pod gets a guaranteed amount of resources available for it

What does one count of CPU really mean? 0.1 can be expressed as 100M where m standard for Milli you can go as low as 1milli

1 count of CPU is equivalent to 1 AWS vCPU , 1 GCP core, 1 Azure Core, 1 Hyperthread j—
Can request a higher level of CPU provided that your nodes are sufficiently funded.

Similarly you can specify the memory using the Mi suffix or specify the memory using the exact amount....

1G - base 10 1Gi - base 2

A container has no limit to the amount of resources it can consume on a node. It can start with one CPU and consume as many resources as it requires and that can suffocate the native process on the node or other containers or resources. You can set a limit of the resource usage on these pods. You can limit it to 1 vCPU or limit it to 512 Mi

Listing 3.36: pod-definition.yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
      - containerPort: 8080
    resources:
      requests:
        memory: "4gi"
        cpu: 2
      limits: # Resources limit
        memory: "2Gi"
        cpu: 2

```

Now when the pod is created, Kubernetes sets new limits for the container. Remember that the limits and requests are set for each container within a pod. If there are multiple containers, then each container can have a request or limited set of its own.

What happens when a pod tries to exceed the resources beyond its specified limit? In case of the CPU, the system throttles the CPU so that it does not go beyond the specified limit. A container cannot use more CPU resources than its limit.

–¿ This is not the case with memory. A container can use more memory resources than its limit, so if a pod tries to consume more memory than its limit constantly. –¿ The pod will be terminated and you will see that with the pod terminated command with an error in the logs. –¿ can also be seen in the output of the describe command when you run it

OOM - refers to an out of memory container kill.

By default kubernetes does not have a CPU or a memory request or limit set. This means that any pod can consume as much resources as required on any node and suffocate other pods or processes that are running on the node, of resources. This is important to not.

Examples scenario: Say there are two pods competing for CPU resources on the cluster. POD/Container. Without a resource or limit set, one pod can consume all the CPU resources on the node and prevent the second part of the required resources.

Can set limits to prevent users from misusing the infrastructure.

Need make sure that pods have request set because that is the only way to guarantee resources.

For memory it is quite similar. Unlike memory we cannot throttle it. Once memory is assigned the only way to free it is to kill it.

Limit Ranges

Limit ranges can help you define default values to be set for containers in pods that are created without a request or limit specified in the Pod definition files. This is applicable at the namespace level.

Listing 3.37: Limit range example

```
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-resource-constraint
spec:
  limits:
  - default:
      cpu: 500m
    defaultRequest:
      cpu: 500m
    max:
      cpu: "1"
    min:
      cpu: 100m
    type: Container
```

Listing 3.38: Limit range example

```
apiVersion: v1
kind: LimitRange
metadata:
  name: memory-resource-constraint
spec:
  limits:
```

```
- default:
  cpu: 1Gi
  defaultRequest:
    cpu: 1Gi
  max:
    cpu: 1Gi
  min:
    cpu: 500Mi
  type: Container
```

These limits are enforced when a pod is created. If you create or change a limit range it does not affect existing pods. It will only affect newer pods that are created after the limit range is created or updated.

There is a way to restrict the total amount of resources that can be consumed by application in a Kubernetes cluster. We can create quotas at a namespace level. So a resource quota is a namespace level object that can be created to set hard limits for requests and limits.

Listing 3.39: Limit range example

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: my-resource-quota
spec:
  hard:
    request.cpu: 4
    request.memory: 4Gi
    limits.cpu: 10
    limits.memory: 10Gi
```

3.10 Service Account

The concept of service accounts is related to other security-related concepts in Kubernetes such as authentication authorization and RBAC etc.

There are two types of account in Kubernetes a user account and a service account. The user account is used by humans and the service accounts are used by machines.

A user account could be for an administrator accessing the cluster to perform administrative tasks, or a developer access the cluster to deploy applications etc.

A service account could be an account used by an application to interact with the Kubernetes cluster. E.g a monitoring application like Prometheus uses a service account to pull the Kubernetes API for performance metrics. An automated build tool like Jenkins uses service account to deploy applications in the Kubernetes cluster.

Listing 3.40: Commands

```
kubectl create serviceaccount <NAME> # use the create a service account on the
kubectl get serviceaccount # view service accounts

kubectl describe serviceaccount <NAME>
```

When the service account is created it also creates a token automatically. The service account token is what must be used by the external applications while authenticating to the Kubernetes API. The token however is stored as a secret object.

When a service account is created it first creates the service account objects, then it generates a token for the service account. It then creates a secret object and stores that token inside the secret objects. The secret object is then linked to the service account.

To view the token, view the secret object by running...

Listing 3.41: Commands

```
kubectl describe serviceaccount <NAME>
```

This token can then be used as an authentication bearer token while making a REST call to the Kubernetes API

You can create a service account and assign the write permission using role-based access control mechanisms and export your service account tokens and use it to configure your third party application to authenticate to the Kubernetes API.

Consider if the third-party application is hosted on the Kubernetes cluster itself? In that case, this whole process of exporting the service account token and configuring the third party application to use it can be made simple by automatically mounting the service token secret as a volume inside the pod hosting the third-party application. That way the token to access the Kubernetes API is already placed inside the pod and can be easily read by the application. You don't have to provide it manually.

For every namespace in Kubernetes, a service account named default is automatically created. Each namespace has its own default service account. Whenever a pod is created the default service account and its token are automatically mounted to that pod as a volume mount.

The default service account has restrictions. It only has the permission to run basic Kubernetes API queries. To use a different service account, modify the pod definition file to include a service account field and specify the name of the new service account. Note: You cannot edit the service account of an existing pod. You must delete and recreate the pod.

In case of deployment, you will be able to edit the service account as any changes to the pod definition file will automatically trigger a new roll out for the deployment. The deployment will take care of deleting and recreating new pods with the right service account.

Kubernetes automatically mounts the default service account if you haven't specified any. You may choose not to mount a service account automatically by setting the `automountServiceAccountToken` field to false in the pod section.

Changes in 1.22 and 1.24 of Kubernetes

Every namespace has a default service account. That service account has a secret object with a token associated with it. When a pod is created, it automatically associates the service account to the pod and mounts the token to a well-known location within the pod.

This makes the token accessible to a process that's running within the pod, and that enables the process to query the Kubernetes API.

Each JWT requires a separate secret object per service account, which results in scalability issues. In version 1.22 the TokenRequest API was introduced as part of the Kubernetes Enhancement Proposal 12005 that aimed to introduce a mechanism for provisioning Kubernetes service account tokens that are more secure and scalable via an API Token generated by the token request API. Audience bound, they're time bound and object bound.

Changes in v1.24 With version 1.24, a change was made where when you create a service account it no longer automatically creates a secret or a token access secret. You must run the

command `kubectl create token` followed by the name of the service account to generate a token for that service account if you needed one.

NOTE: see kubernetes service account secrets

3.11 Taints and Tolerations

Taints and tolerations have nothing to do with security or intrusion on the cluster. Taints and toleration are used to set restrictions on what pods can be scheduled on a node.

Example: Consider a cluster where a node contains dedicated resources. We do not want the scheduler to put any pod on that node. This can be achieved using a taint. Pods have no tolerations by default, so they cannot tolerate any taint.

One half is tainting the node. The other is specifying which pods are tolerant to the taint.

Taints are set on nodes. Tolerations are set on pods.

Listing 3.42: Commands

```
kubectl taint nodes node-name key=value:taint-effect # taint effect defines wh
```

```
kubectl taint nodes node1 app=myapp:NoSchedule
```

Three types of taint effect

- No schedule, which means the pods will not be scheduled on the node.
- Preferred no schedule, which means that the scheduler/system will try to avoid placing a pod on the node but that is not guaranteed.
- Thins i no execute, which means that the new pods will not be scheduled on the node and existing pods on the node if any will be evicted if they do not tolerate the taint. These pods may have been scheduled on the node before the taint was applied to the node.

Tolerations - Pods

Listing 3.43: Limit range example

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: nginx-container
    image: nginx
  tolerations:
  - key: "app"
    operator: "Equal"
    value: "blue"
    effect: "NoSchedule"
```

Listing 3.44: Commands

```
kubectl taint nodes node1 app = blue : NoSchedule
```

When the pods are not created or updated with the new tolerations, they are either not scheduled on nodes or evicted from the existing nodes depending on the effect set.

No Execute taint effect

If a pod does not have a tolerance it is evicted i.e killed.

Taints and tolerations do not tell a particular pod to go to a particular node.

Instead it tell the node to accept pods with certain tolerations.

If the requirement is to restrict a pod to certain nodes it is achieved through the concept of node affinity.

Master nodes

Taint is automatically applied on this node when the cluster is setup that prevent any pods from being scehduled on this node. This behaviour can be modified if required but it is best practice not to deploy applicatn workloads on a master server.

Listing 3.45: Commands

```
kubectl describe node kubemaster | grep taint
```

3.12 Node Selectors Logging

We can set a limitation on the pods so that they only runb on particular nodes. There are two ways to do this. The first is using node electores. For this we look at the pod defintion file.

Listing 3.46: Node selector example

```
apiVersion:
kind: pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor
  nodeSelector:
    size: Large
```

Kubernetes nodes which nodes are which via key value pairs which are assigned to the node. The scehduler uses the labels to identify the right node.

To use labels in a node elector like this, you must have first labeled your nodes prior to creating this pod.

Listing 3.47: Commands

```
kubectl label nodes <node-names> <label-key>=<label-value>

kubectl label nodes node-1 size=Large
```

Node slectors have limitations. Consider more complext requirements Node selectors do not allow for more complex logic. For more complextr logic node affinity and anti-affinity features were introduced.

3.13 Node Affinity

The primary purpose of node affinity feature is to ensure that pods are hosted on particular nodes. This feature allows us with advanced capabilities to limit pod placement on specific nodes.

Listing 3.48: Node affinity example

```
apiVersion:
kind: pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: Size
            operator: In #Can also use the noIn operator
            values: # A list of values
          - Large
          - Medium
```

Check documentation for other operators.

Consider if a node affinity could not match a node with a given expression.

Node Affinity Types

The type of node affinity defines the behaviour of the scheduler with respect to node affinity and the stages in the lifecycle of the pod.

There are currently two types of node affinity available.

`requiredDuringSchedulingIgnoredDuringExecution` `preferredDuringSchedulingIgnoredDuringExecution`

There are two states in the life cycle of a pod when considering node affinity during scheduling and during execution.

Consider that the nodes with matching labels are not available? This is where the type of node affinity available comes into play.

If you select the required type, which is the first one the scheduler will mandate that the pod be placed on a node with the given affinity rules. If it cannot find one the pod will not be scheduled. This type will be used in cases where the placement of the pod is crucial. If a matching node does not exist the pod will not be scheduled.

What if the pod placement is less important than running the workload itself? In that case you could set it to preferred. And in cases where a matching node is not found the scheduler will simply ignore ignored node affinity rules and place the pod on any available node.

Effectively the scheduler will try best to place the pod on a matching node but if none are available place it anywhere.

The second part of the property or the other state is during execution. During execution is the state where a pod has been running and a change is made in the environment that affects node affinity such as a change in the label of a node.

In the case what would happen to the pods is that value is set to ignored. I.e this means pods will continue to run and any changes in the node affinity will not impact them once they are scheduled.

3.14 Taints and Tolerations vs Node Affinity

A combination of taints and tolerations and node affinity rules can be used together to completely dedicate nodes for specific pods.

Chapter 4

Multi-container pods

4.1 Multi-container pods

4.2 Multi-container pods design patterns

4.3 Init containers

Chapter 5

Observability

5.1 Readiness and Liveness Probes

5.2 Liveness Probes

5.3 Logging

5.4 Monitor and Debug application

Chapter 6

Pod Design

6.1 Labels, Selectors and Annotations

6.2 Rolling updates and Rollbacks in Deployments

6.3 Updating a deployment

6.4 Deployment strategy - Blue Green

6.5 Deployment strategy - Canary

6.6 Jobs

6.7 CronJobs

Chapter 7

Services and Networking

7.1 Network Policies

7.2 Developing network policies

7.3 Ingress networking

Chapter 8

State Persistence

- 8.1 Introduction to Docker Storage**
- 8.2 Storage in Docker**
- 8.3 Volume Driver Plugins in Docker**
- 8.4 Volumes in Kubernetes**
- 8.5 Persistent Volumes**
- 8.6 Persistent volume claims**
- 8.7 Using PVCs in Pods**
- 8.8 Storage Classes**
- 8.9 Why Stateful Sets**
- 8.10 Stateful sets introduction**
- 8.11 Headless Services**
- 8.12 Storage in StatefulSets**

Chapter 9

Security

9.1 Kubernetes Security Primitives

9.2 Authentication

9.3 KubeConfig

9.4 API Groups

9.5 Authorization

9.6 Role based access controls

9.7 Cluster Roles

9.8 Admission Controllers

9.9 Validating and Mutating Admission Controllers

9.10 API Versions

9.11 API Deprecations

9.12 Custom Resource Definition

9.13 Custom Controllers

9.14 Operator Framework

Chapter 10

Helm Fundamentals

10.1 Helm Introduction

10.2 Install Helm

10.3 Helm concepts

Chapter 11

Kustomize

11.1 Kustomize Problem Statement and Ideology

11.2 Kustomize vs Helm

11.3 Kustomization yaml file

11.4 Kustomization Output

11.5 Kustomization ApiVersion and Kind

11.6 Managing Directories

11.7 Common Transformers

11.8 Image Transformers

11.9 Patches Intro

11.10 Different Types of Patches

11.11 Patches Dictionary

11.12 Patches list

11.13 Overlays

11.14 Components